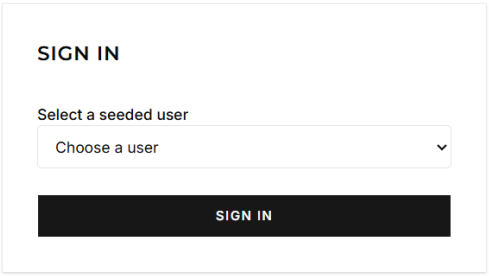
STORY 1

Log In As A Seeded User

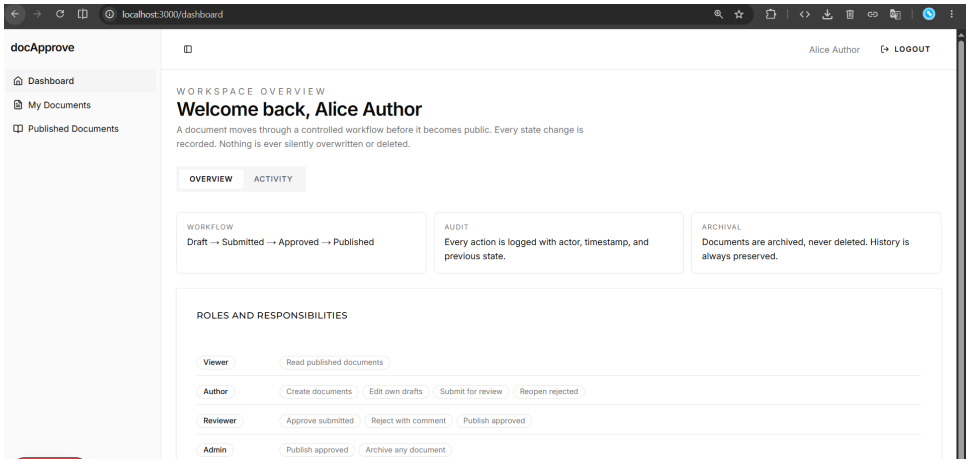
Proves that authentication is enforced server-side — not just hidden in the UI. The server rejects unauthenticated requests (401) and wrong-role requests (403) regardless of what the client sends.

Evidence 1 — Login page: seeded user dropdown



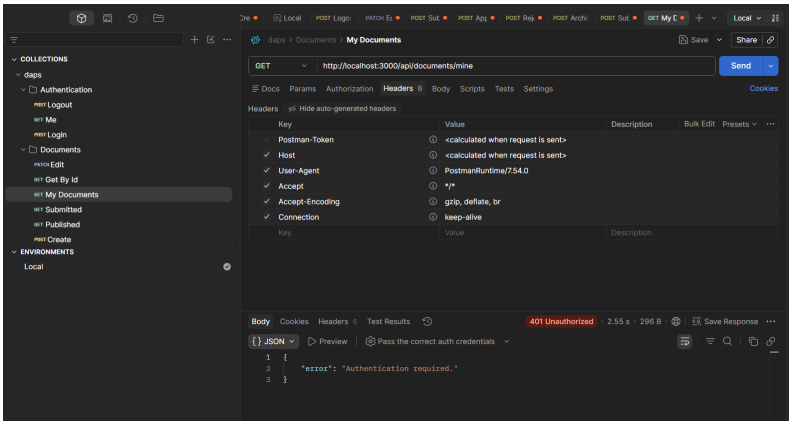
The login page presents all 5 seeded users in a dropdown. No password required — selecting a user creates a server-side session.

Evidence 2 — Logged-in header showing name and role



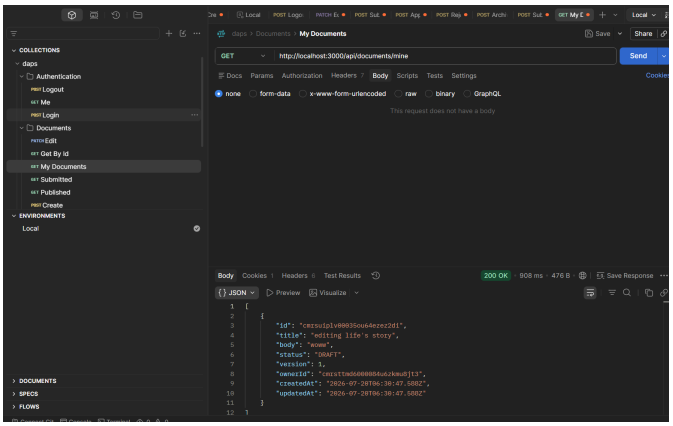
After logging in as Alice Author, the header shows 'Alice Author' and the sidebar shows only Author-scoped routes (My Documents, Published Documents).

Evidence 3 — GET /api/documents/mine without cookie → 401



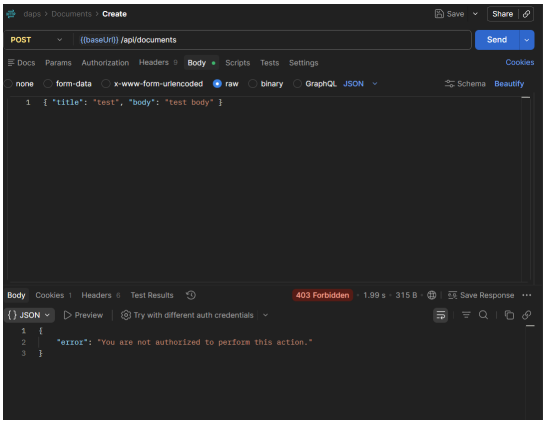
No Cookie header sent. Server returns 401 with { "error": "Authentication required." }.

Evidence 4 — Same route with valid session cookie → 200 OK



With a valid session cookie, the server resolves the user and returns the document list. Status 200 OK.

Evidence 5 — POST /api/documents as Viewer (wrong role) → 403



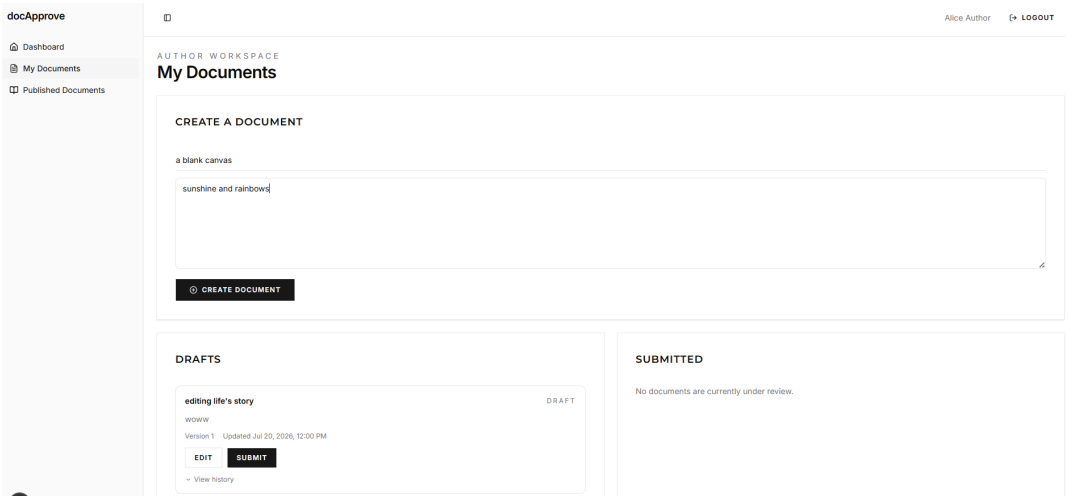
Viewer's session cookie sent. Server calls assertAuthor(), which throws AuthorizationError → 403 { "error": "You are not authorized to perform this action." }. The UI button never even existed for this user.

STORY 2

Create A Draft Document

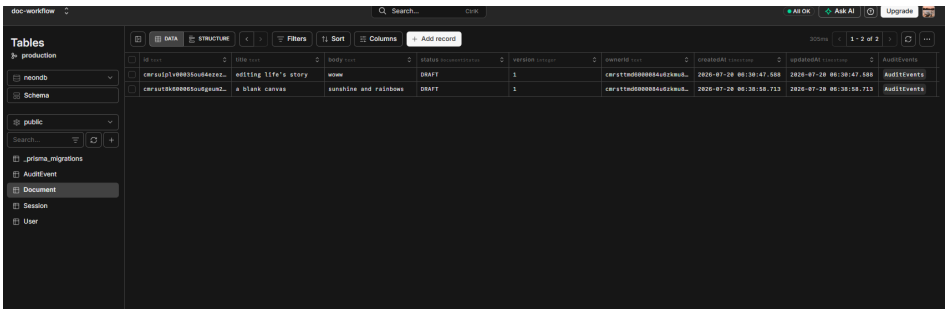
Proves that document creation sets status=DRAFT and ownerId from the session, that empty title/body is rejected server-side, and that a CREATED audit event is written atomically with the document row.

Evidence 1 — Create Document form and resulting draft in the UI



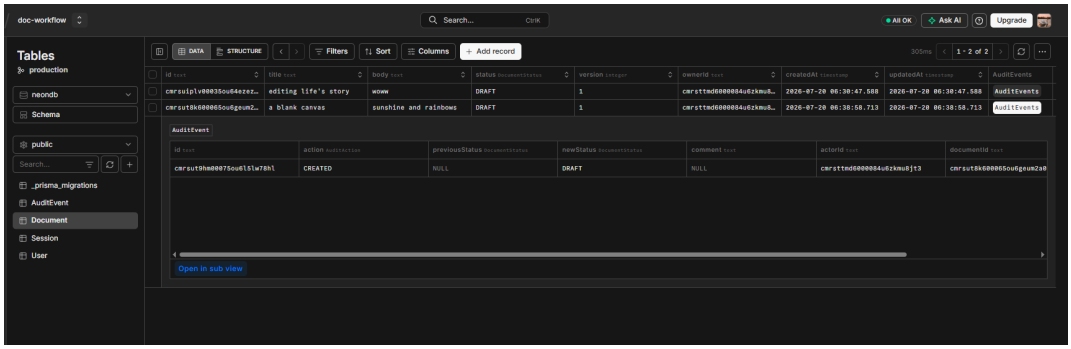
Title 'a blank canvas' and body 'sunshine and rainbows' entered. After clicking CREATE DOCUMENT the draft appears immediately in the Drafts section.

Evidence 2 — Prisma Studio: Document row (status=DRAFT, ownerId set)



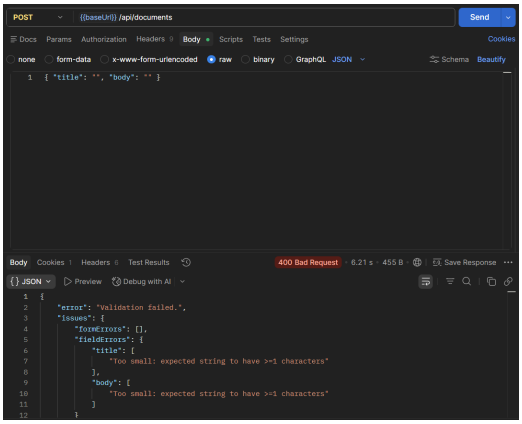
Both documents show status=DRAFT, version=1, and ownerId populated with Alice's user ID.

Evidence 3 — Prisma Studio: AuditEvent (action=CREATED, newStatus=DRAFT)



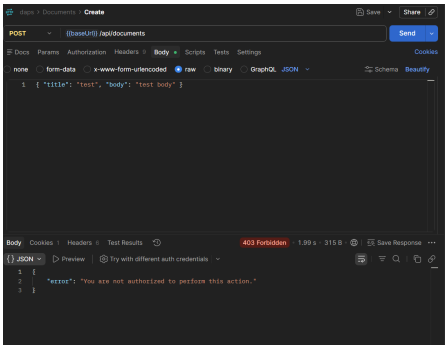
Inline AuditEvent for 'a blank canvas': action=CREATED, previousStatus=NULL, newStatus=DRAFT. Written atomically with the Document row inside db.$transaction.

Evidence 4 — POST /api/documents with empty title+body → 400 Bad Request



Zod schema (createDocumentSchema) enforces min(1) on both fields. Returns 400 with fieldErrors on title and body.

Evidence 5 — POST /api/documents as Viewer → 403 (reused from Story 1 Evidence 5)



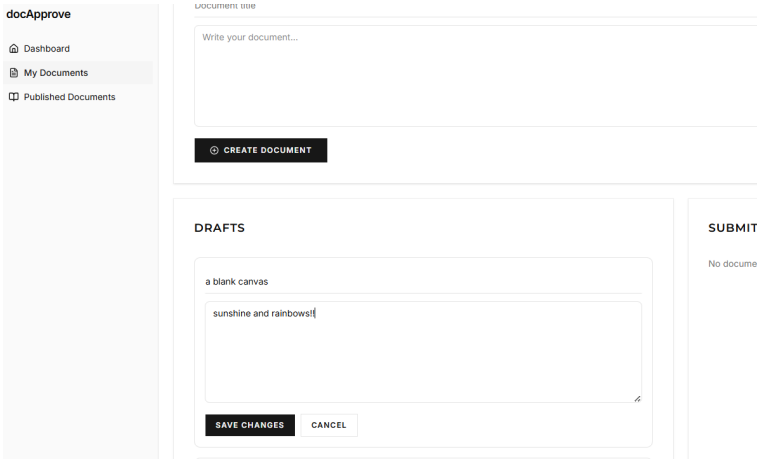
assertAuthor() in documentService.create() blocks the Viewer before any DB operation is attempted.

STORY 3

Edit A Draft Document

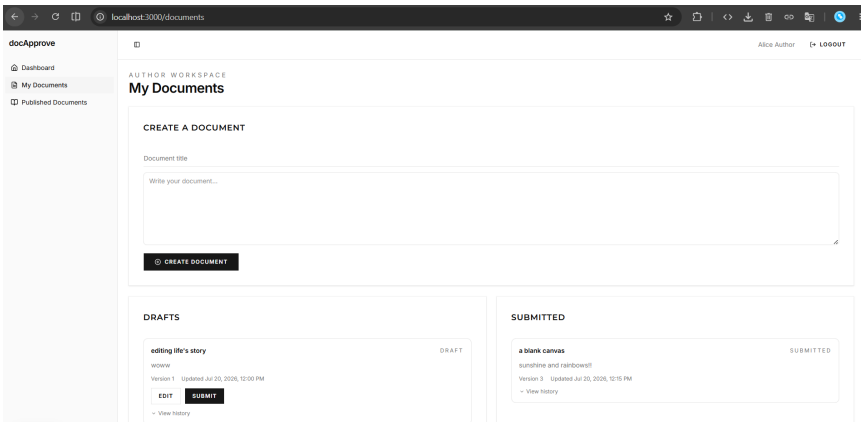
Proves that only the document owner can edit, only DRAFT documents are editable, an EDITED audit event is written on every save, and version increments.

Evidence 1a — Edit form open on 'a blank canvas'



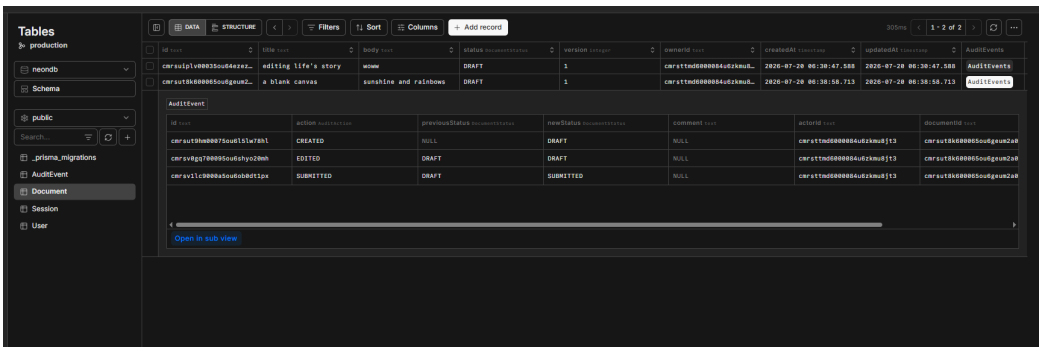
Alice clicks Edit. The inline form shows the current title and body with SAVE CHANGES / CANCEL controls.

Evidence 1b — After save: document moved to Submitted at Version 3



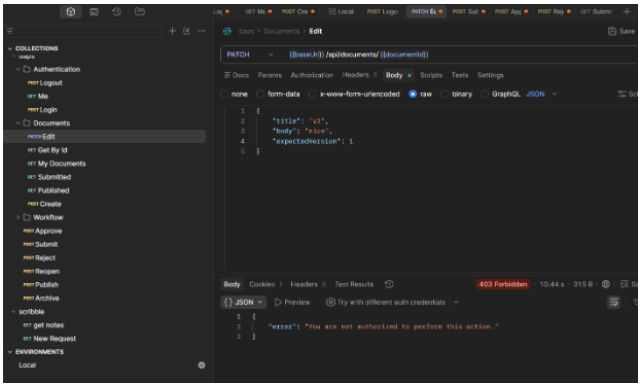
Document 'a blank canvas' now appears in the Submitted column at Version 3 (edited once → version 2, submitted → version 3), confirming version increments on every mutation.

Evidence 2 — Prisma Studio: AuditEvent chain (CREATED → EDITED → SUBMITTED)



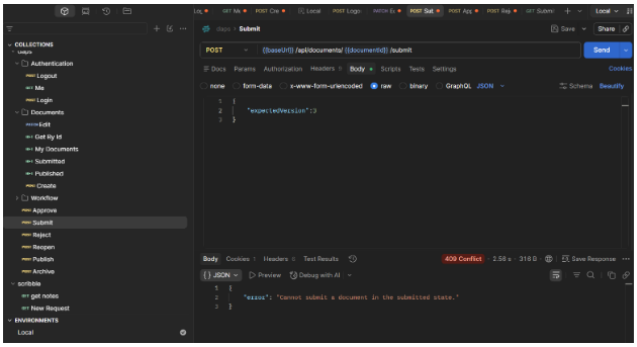
Three events on 'a blank canvas': CREATED (null→DRAFT), EDITED (DRAFT→DRAFT), SUBMITTED (DRAFT→SUBMITTED). All share the same documentId.

Evidence 3 — PATCH /api/documents/:id as Alice on Ashton's doc → 403



assertOwner() check in documentService.edit() fires before any update. 403 returned even though Alice is a valid Author.

Evidence 4 — PATCH on already-SUBMITTED document → 409 Conflict



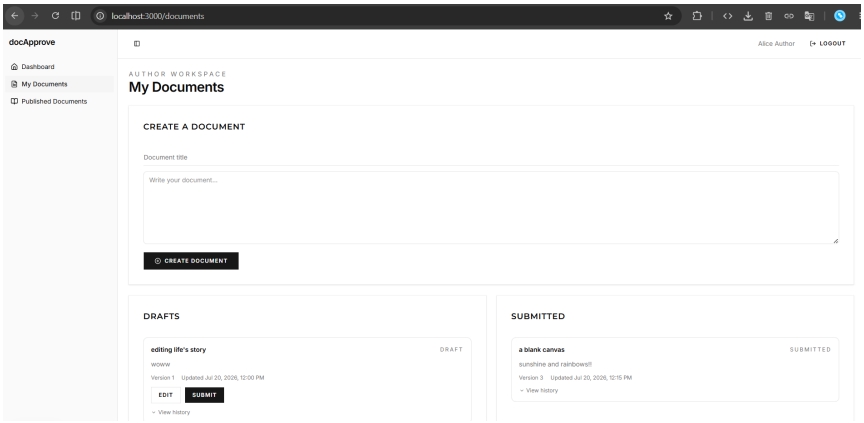
assertDraft() fires for the submit action (same pattern for edit). 409 { "error": "Cannot submit a document in the submitted state." }.

STORY 4

Submit A Document For Review

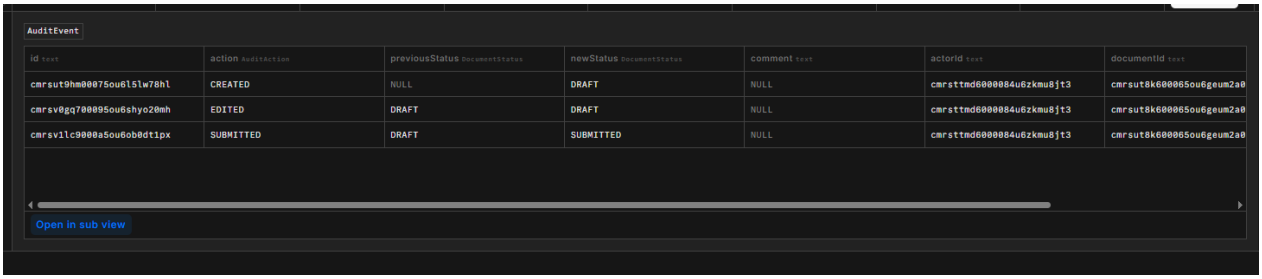
Proves the DRAFT→SUBMITTED transition is validated server-side, the document appears in the reviewer queue, and double-submission is blocked.

Evidence 1 — Draft visible in Author workspace; after Submit it moves to Submitted column



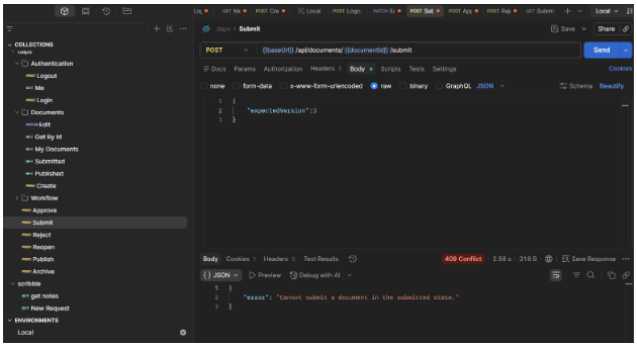
'a blank canvas' appears in the SUBMITTED column at Version 3. The Edit and Submit buttons are no longer shown — document is read-only to the author while submitted.

Evidence 2 — Prisma Studio: Document=SUBMITTED, AuditEvent action=SUBMITTED



AuditEvent row: action=SUBMITTED, previousStatus=DRAFT, newStatus=SUBMITTED. Same actorId as the document owner, written in the same transaction.

Evidence 3 — POST /api/documents/:id/submit on already-SUBMITTED doc → 409



assertDraft() in documentService.submit() throws ConflictError. 409 { "error": "Cannot submit a document in the submitted state." }.

STORY 5

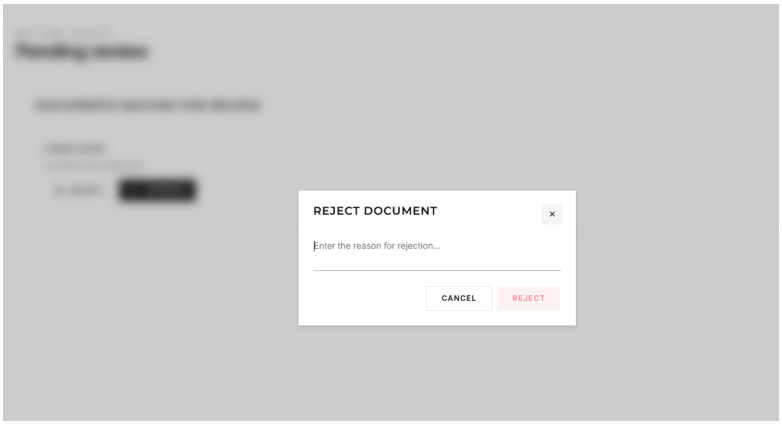
Review A Submitted Document (Approve / Reject)

Proves that only Reviewers can approve/reject, the document owner cannot approve their own document, rejection requires a non-empty comment (enforced at both schema and DB constraint levels), and audit events are written for both actions.

Evidence 1 — Reviewer queue showing submitted document with Approve/Reject buttons

Bob Reviewer sees 'a blank canvas' as SUBMITTED with Reject and Approve buttons. Authors and Viewers do not see this page (requireReviewer() guard on the route).

Evidence 2 — Reject dialog requiring a comment before enabling Reject button

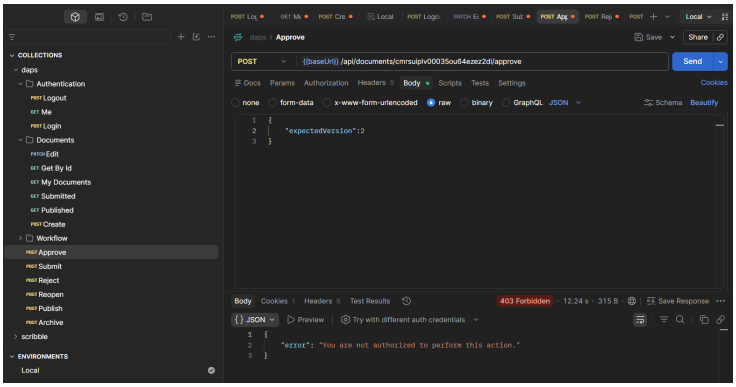


The Reject button is disabled (muted) until the textarea contains text. Even if bypassed in the UI, the server validates comment.trim().length > 0 and the DB has a CHECK constraint on the AuditEvent table.

Evidence 3 — Prisma Studio: REJECTED event with comment populated

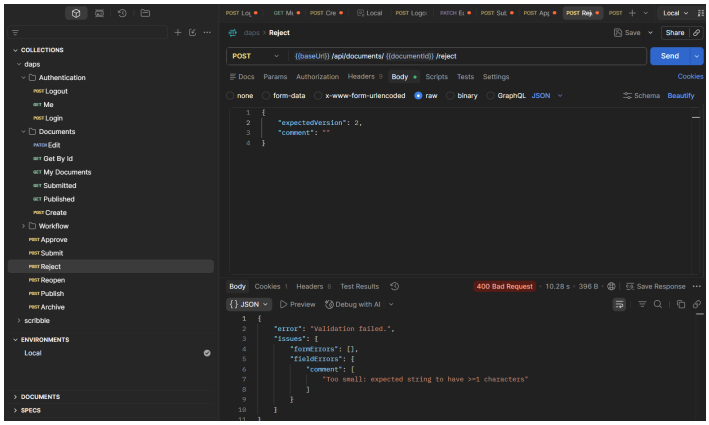
AuditEvent row: action=REJECTED, previousStatus=SUBMITTED, newStatus=REJECTED, comment='no punctuation'. actorId is Bob Reviewer's ID (different from Alice's ownerId).

Evidence 4 — POST /api/documents/:id/approve as document owner (Alice) → 403



Alice's session cookie. documentService.approve() checks document.ownerId === actor.id and throws AuthorizationError. Self-approval is blocked at the service layer.

Evidence 5 — POST /api/documents/:id/reject with empty comment → 400



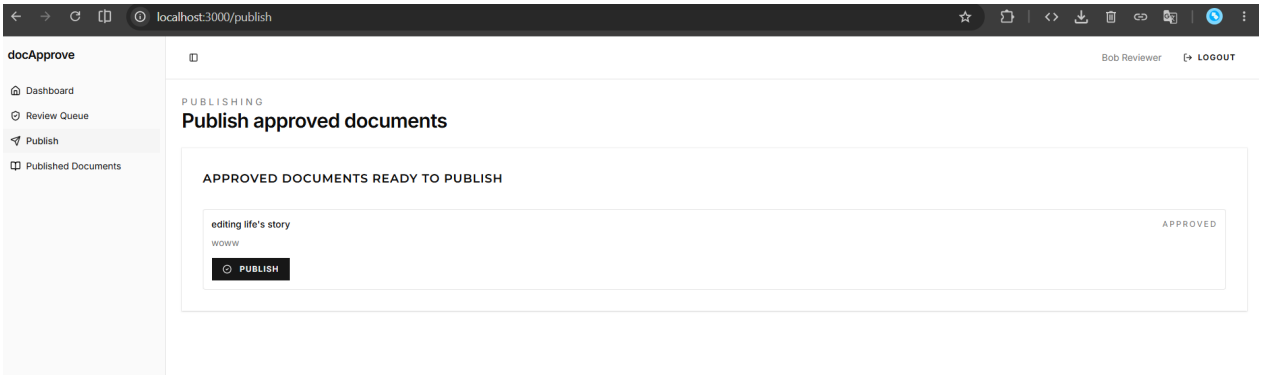
rejectDocumentSchema extends actionDocumentSchema with comment: z.string().trim().min(1). Returns 400 with fieldError: 'Too small: expected string to have >=1 characters'.

STORY 6

Publish An Approved Document

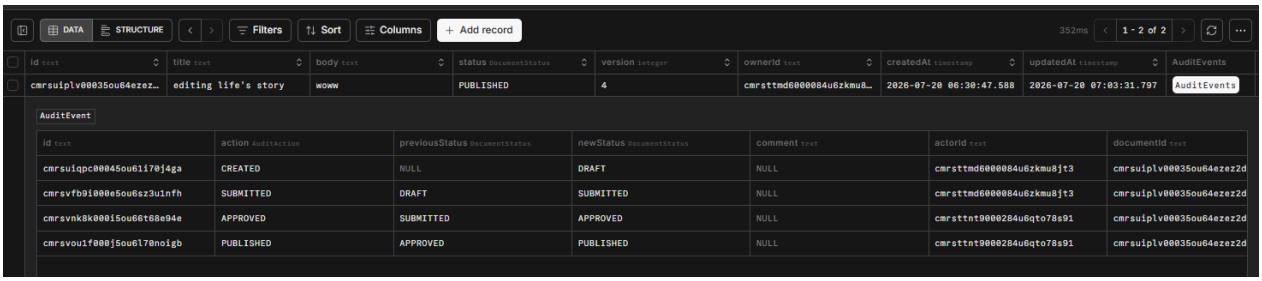
Proves that only APPROVED documents can be published, only Reviewers or Admins can trigger the action, and viewers have no path to non-published documents.

Evidence 1 — Publish page (Bob Reviewer): approved document with Publish button



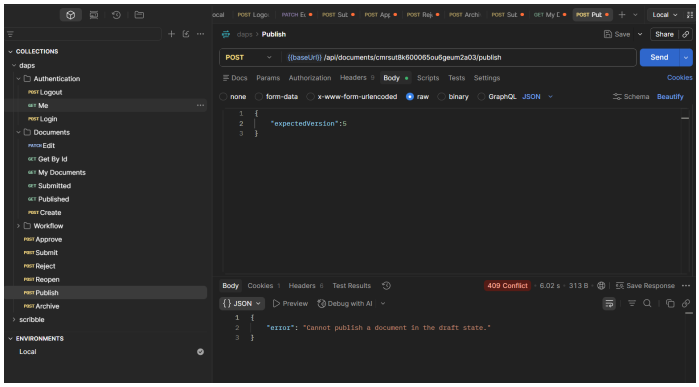
'editing life\'s story' shows as APPROVED on the Publish page. The Publish button triggers POST /api/documents/:id/publish.

Evidence 2 — Prisma Studio: Document=PUBLISHED, AuditEvent action=PUBLISHED



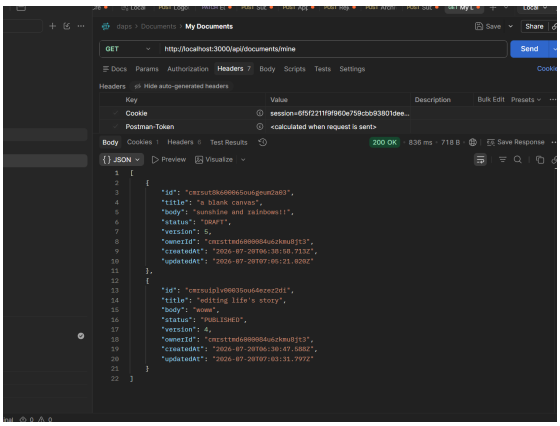
Document row: status=PUBLISHED, version=4. AuditEvent chain: CREATED → SUBMITTED → APPROVED → PUBLISHED (previousStatus=APPROVED, newStatus=PUBLISHED). actorId is Bob Reviewer.

Evidence 3 — POST /api/documents/:id/publish on DRAFT document → 409



assertApproved() in documentService.publish() fires. 409 { "error": "Cannot publish a document in the draft state." }.

Evidence 4 — GET /api/documents/mine with author session: confirms per-user scoping



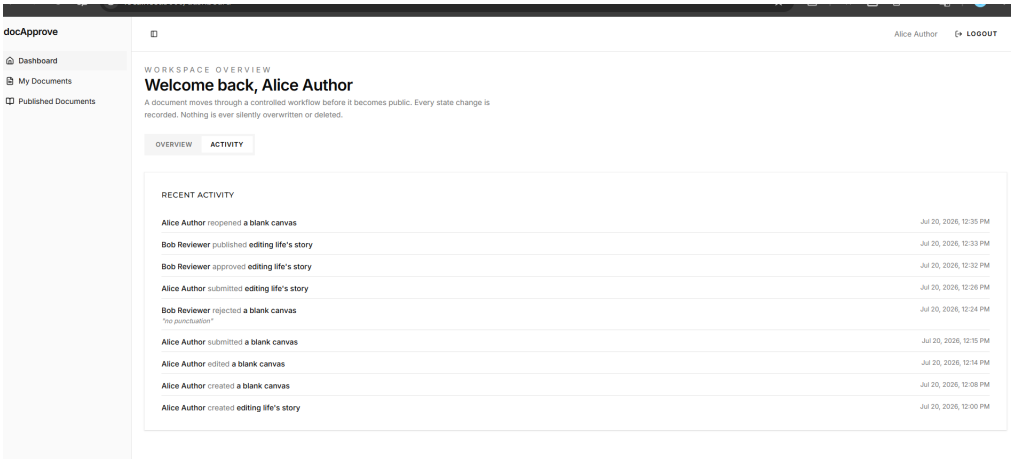
With Alice's session cookie, GET /api/documents/mine returns only Alice's own documents (200 OK, both DRAFT and PUBLISHED). A Viewer calling this same endpoint gets 403 (assertAuthor guard). Viewers are limited to GET /api/documents/published only.

STORY 7

View Audit History

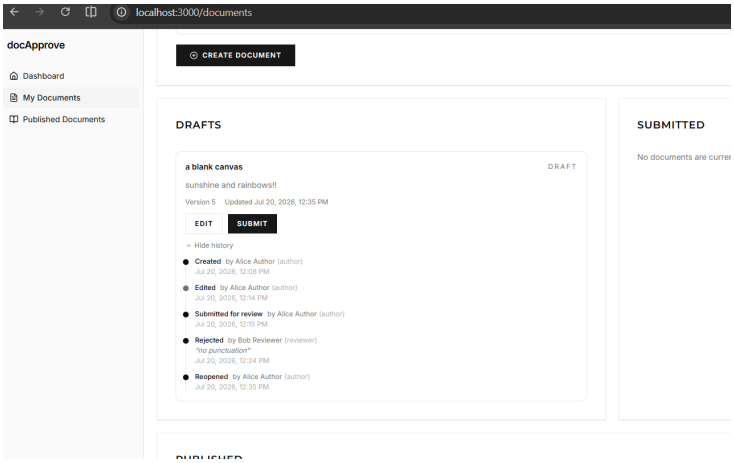
Proves every state change is recorded with actor, action, timestamp, and previous/new status — and that state update + audit write share a single database transaction so they can never drift apart.

Evidence 1 — Dashboard Activity tab: chronological feed across all documents



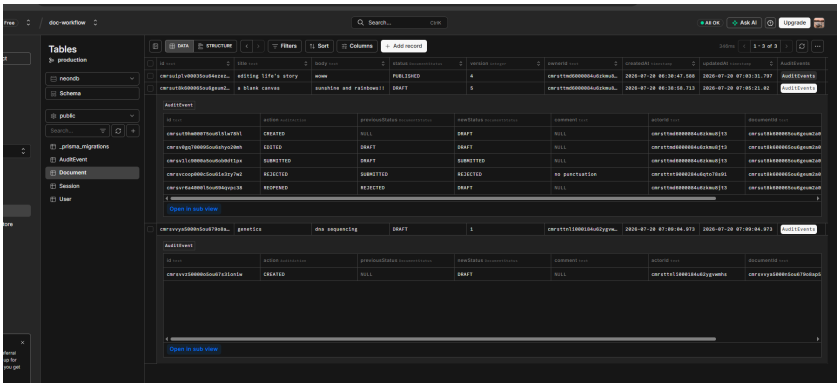
Shows: Alice Author reopened, Bob Reviewer published/approved, Alice submitted/edited/created — each entry shows actor name, action verb, document title, timestamp, and rejection comment where applicable.

Evidence 2 — Inline audit timeline on 'a blank canvas'



Five events shown: Created → Edited → Submitted for review → Rejected ('no punctuation') → Reopened. Each shows actor name, role in parentheses, and timestamp.

Evidence 3 — Prisma Studio: full AuditEvent table for 'a blank canvas'



Five AuditEvent rows in ascending createdAt order: CREATED, EDITED, SUBMITTED, REJECTED (comment='no punctuation', different actorId), REOPENED. Matches the UI display exactly.

Transaction implementation

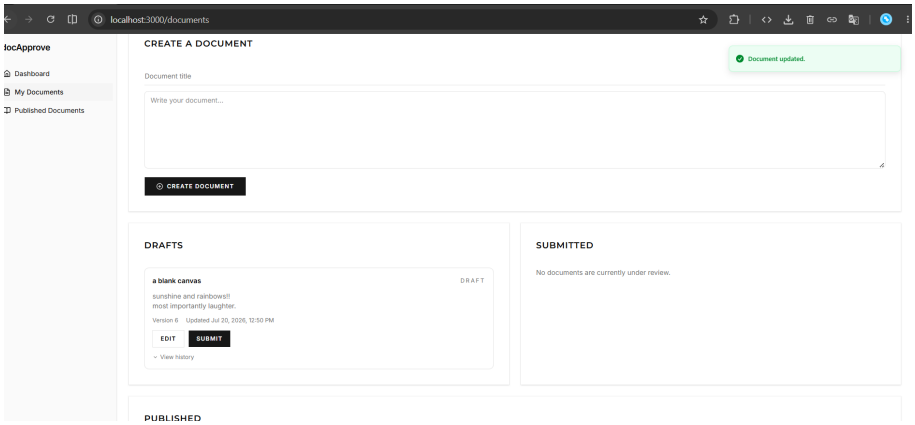
Every write function in **src/services/document.service.ts** (*create, edit, submit, approve, reject, reopen, publish, archive*) wraps both **documentRepository.update()** and **auditRepository.create()** inside a single **db.$transaction(async (tx) => { ... })** block. Both repositories are instantiated with the transaction client **tx**, so if either write fails, the entire operation rolls back atomically. A document can never change state without a matching audit event, and an audit event can never exist for a state change that did not commit.

STORY 8

Handle Concurrent Updates

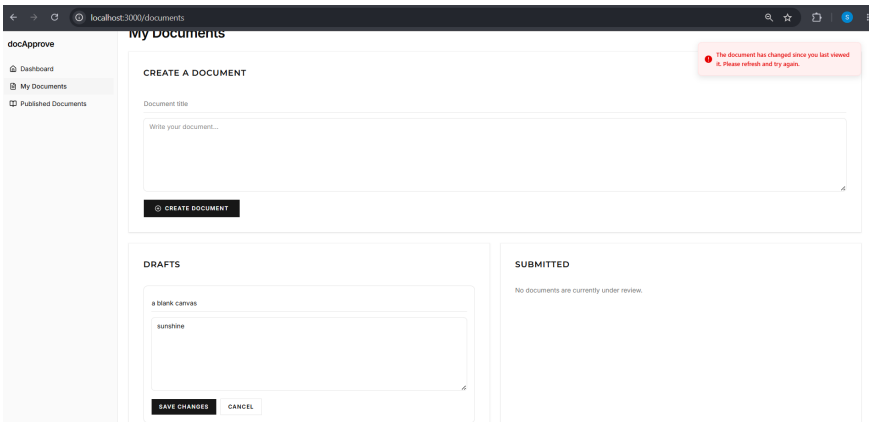
Proves that optimistic concurrency control prevents stale writes. Every mutation sends expectedVersion; the repository uses updateMany WHERE version=expectedVersion. If the version has moved on, count===0 and a 409 ConflictError is returned.

Evidence 1a — Tab 1: successful save, document advances to Version 6



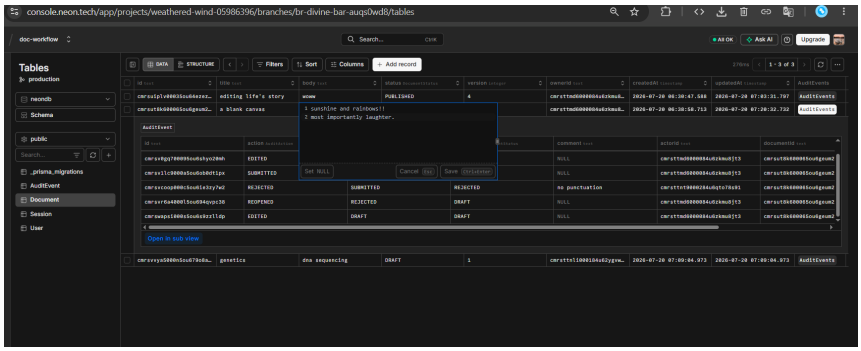
Tab 1 saves first. Green toast: 'Document updated.' Document now at Version 6 with body 'sunshine and rainbows!! most importantly laughter.'

Evidence 1b — Tab 2: stale page tries to save → conflict toast



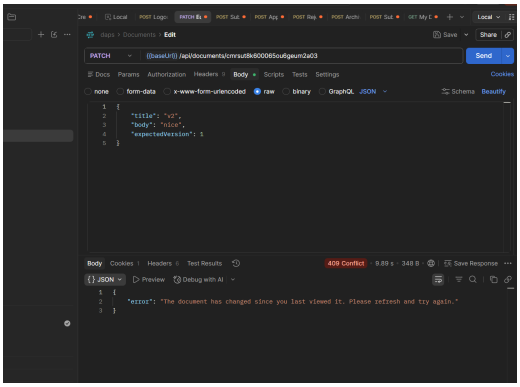
Tab 2 still holds version 5. Its SAVE CHANGES request is rejected. Red error toast: 'The document has changed since you last viewed it. Please refresh and try again.'

Evidence 2 — Prisma Studio: version column at 6, only one write landed



'a blank canvas' at version=6. The AuditEvent chain shows two EDITED rows (one per successful save), confirming exactly one write per version increment and no silent overwrite.

Evidence 3 — PATCH with stale expectedVersion:1 (doc is at version 6) → 409



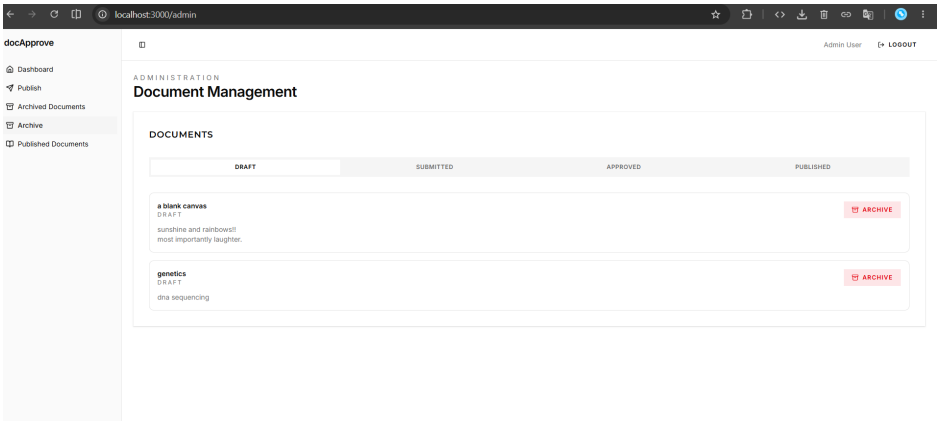
documentRepository.update() uses updateMany WHERE { id, version: 1 }. No row matches → count===0 → ConflictError thrown. 409 { "error": "The document has changed since you last viewed it. Please refresh and try again." }.

STORY 9

Archive A Document

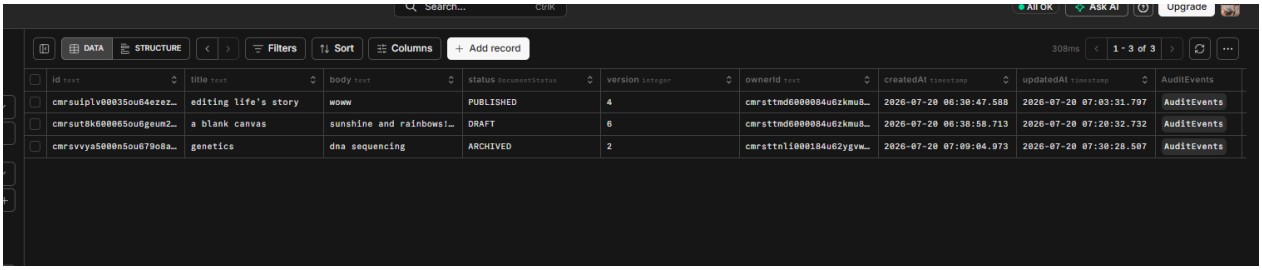
Proves that Admins can archive, archiving is a soft delete (row preserved), an ARCHIVED audit event is written in the same transaction, and archived documents cannot be further mutated.

Evidence 1 — Admin panel: Document Management with Archive buttons



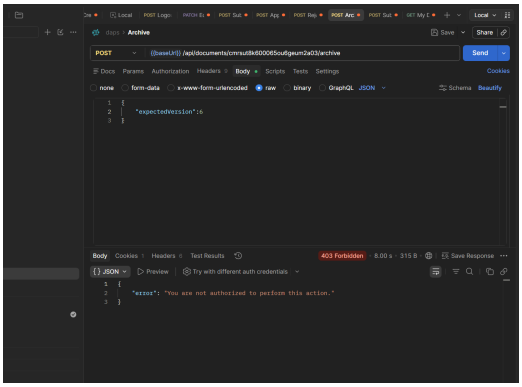
Admin User sees DRAFT, SUBMITTED, APPROVED, and PUBLISHED tabs. Each document has a red Archive button. This page is guarded by requireAdmin().

Evidence 2 — Prisma Studio: 'genetics' row status=ARCHIVED, row still exists



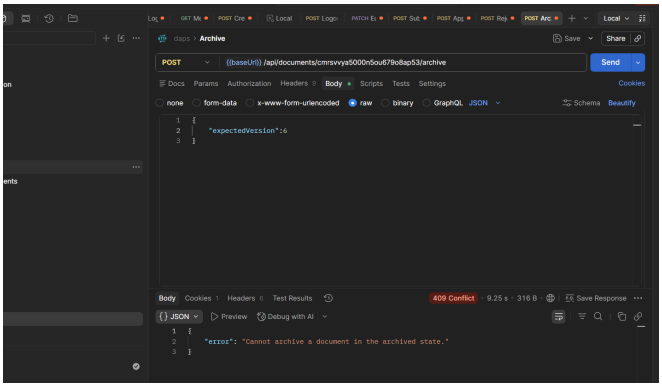
The 'genetics' row is present in the Document table with status=ARCHIVED, version=2. Not deleted — history and the document record are fully preserved.

Evidence 3 — POST /api/documents/:id/archive as non-Admin → 403



Alice's session cookie (AUTHOR role). assertAdmin() in documentService.archive() throws AuthorizationError. 403 { "error": "You are not authorized to perform this action." }.

Evidence 4 — POST /api/documents/:id/archive on already-ARCHIVED doc → 409



assertArchivable() allows only DRAFT/SUBMITTED/APPROVED/PUBLISHED → ARCHIVED. Attempting to re-archive returns 409 { "error": "Cannot archive a document in the archived state." }.